

## Hog 游戏实验报告

实验概述 本项目为 CS61A Hog 骰子游戏编程实验，共包含 12 道编程习题。我完成所有 # BEGIN PROBLEM 至 # END PROBLEM 区间内核心代码的编写、边界调试与验证，实现 Hog 游戏规则、对局模拟器与多套博弈策略，代码可正常运行。

本人完成的 12 道题目

Phase1: 游戏模拟器 (Problem1 ~ Problem5)

Problem1 roll\_dice: 骰子模拟，投掷出现 1 则本轮得 1 分，否则返回点数总和。

Problem2 boar\_brawl: 实现投 0 颗骰子的 Boar Brawl 得分规则。

Problem3 take\_turn: 整合单回合逻辑，根据骰子数量调用对应得分函数，校验参数范围。

Problem4: 编写质数、因数统计函数，实现 Sus Fuss 特殊分数修正与分数更新逻辑。

Problem5 play: 双人对局模拟器，玩家交替行动，直到一方分数达标，输出最终对局分数。

Phase2: 玩家策略与性能评估 (Problem6 ~ Problem12)

Problem6 always\_roll(n): 高阶函数，生成固定投掷 n 颗骰子的策略。

Problem7 is\_always\_roll: 遍历全部分数状态，检测策略是否始终返回相同骰子数量。

Problem8 make\_averaged: 高阶闭包，借助 \*args 可变参数，多次调用目标函数并返回均值，用于随机结果降噪。 Problem9 max\_scoring\_num\_rolls: 遍历 1~10 颗骰子，利用均值函数对比期望得分，选出最优投骰子数量。

Problem10 boar\_strategy: Boar Brawl 策略，评估 0 骰子收益，达到阈值则投 0 颗骰子。

Problem11 sus\_strategy: 综合 Boar Brawl、Sus Fuss 两套规则，判断是否选择 0 颗骰子。

Problem12 final\_strategy: 自定义博弈策略，依据双方分数动态选择每轮投掷骰子数量。

## 调试过程与完成程度

开发中我针对循环逻辑、高阶函数、可变参数、边界条件反复调试，保证代码稳定运行。全部代码支持完整游戏仿真，可通过项目自带测试用例。

实验小结 本次实验掌握 Python 分支、循环、高阶函数语法，学习回合模拟器开发、随机采样均值计算与博弈策略设计评估的基本思路。

```
• >>> from hog import *
• >>> run_experiments()
Max scoring num rolls for six-sided dice: 6
always_roll(6) win rate: 0.52
catch_up win rate: 0.5365
always_roll(3) win rate: 0.36150000000000004
always_roll(8) win rate: 0.46199999999999997
boar_strategy win rate: 0.4955
sus_strategy win rate: 0.48950000000000005
final_strategy win rate: 0.5045
○ >>> 
```

实验结果也显示在扔六个色子时赢的概率最大。

```
• >>> dice = make_test_dice(4, 2, 5, 1)
... averaged_dice = make_averaged(roll_dice, 40)
... averaged_dice(1, dice)
...
3.0
• >>> play(always_roll(6), always_roll(6), simple_update)
(100, 99)
• >>> play(always_roll(6), always_roll(6), sus_update)
(120, 89)
• >>> catch_up(9, 4)
5
• >>> strategy = always_roll(3)
... strategy(0, 0)
...
3
• >>> dice = make_test_dice(1, 6)
• >>> max_scoring_num_rolls(dice)
1
• >>> is_always_roll(always_roll_5)
True
• >>> is_always_roll(catch_up)
False
• >>> dice = make_test_dice(1, 6)
• >>> max_scoring_num_rolls(dice)
1
○ >>> 
```

各个策略均能运行